SCoPe: Viva Preparation

Everything you need to explain the project, in plain words

1. The project in 30 seconds

Learn this paragraph by heart. If you can say it confidently, you can answer most questions.

> A robot can plan by *imagining*: it tries 300 possible action sequences in its head, using a learned **world model** called DINO-WM, and picks the one whose imagined result is closest to the goal. The problem is that the imagination is sometimes wrong, and the robot confidently does something useless. **Our idea, SCoPe:** before trusting an imagined future, check how reliable it is, and throw away plans the model is unsure about. **What we found:** (1) we got DINO-WM running on our laptop GPU and matched the published results (90%, 95%, 100%); (2) a simple trick, running the model four times with dropout switched on and checking whether the answers disagree (MC-dropout), detects the wrong predictions with an AUROC of 0.86, against 0.79 for a simple baseline and 0.50 for random guessing; (3) removing bad plans only at the very end barely helps, so the check must be used throughout the planning.

2. The story in five steps (about 3 minutes)

1. **Background.** In model-based RL, the agent learns a world model, a network that predicts what happens after an action. It then plans by trying actions in imagination. DINO-WM does this from camera images: it turns each image into features with a pretrained encoder (DINOv2) and predicts how the features change.
2. **Problem.** The planner trusts every imagined future equally. When the model is wrong, the robot fails. Show Fig. 1 of the report: the model imagines the block reaching the goal; in reality it barely moves.
3. **Idea.** Give each imagined step a trust score. Reject plans with an untrusted step. Choose only among trusted plans. "Trust region" means: the situations where the model's predictions are reliable.
4. **What we did.** (a) Reproduced DINO-WM on three tasks on an 8 GB GPU, 3.8× faster. (b) Measured the *true* error of 76,800 imagined steps by executing the plans in the simulator. (c) Compared five trust scores. (d) Tested removing untrusted plans.
5. **Results and lesson.** MC-dropout is the best trust score (AUROC 0.86). The planner's own confidence tells you nothing about errors. Removing plans only at the final choice does not help, even with perfect knowledge of the errors, so the check has to guide the whole search.

3. Numbers to remember

What	Our result	Compare with
Push-T success after replanning	0.90 (3 seeds)	paper 0.90
Wall success after replanning	0.95 (3 seeds)	paper 0.96
PointMaze success after replanning	1.00 (1 seed)	paper 0.98
Push-T success with a single plan	0.87	paper 0.86
Wall success with a single plan	0.52	paper 0.74 (weak point)
Speed-up of our code	3.8× (26.2 s → 6.9 s)	same plans chosen
Imagined steps checked against reality	76,800	= 80 pairs × 192 plans × 5 steps
MC-dropout AUROC	0.86 (interval 0.84–0.88)	best of five scores
Simple baseline (distance moved)	0.79	
Random guessing	0.50	
Planner's own cost as a score	0.51	no better than random
Chosen plan succeeds, no removal	71%	
Chosen plan succeeds, oracle removal	73%	barely different

4. Every term in one line

Reinforcement learning (RL)
An agent learns to act by interacting with an environment to reach a goal.

Model-based RL
The agent learns a model of the environment and uses it to plan, instead of learning only by trial and error.

World model
A neural network that predicts what happens next if the agent takes an action: the robot's imagination.

Latent state
The compact feature vector that describes an image. The world model works on these, not on pixels.

Encoder / DINOv2
A pretrained image network that turns an image into features. It is *frozen*: we never change it.

Predictor
The part of DINO-WM that predicts the next latent state from the current one and an action.

JEPA
Joint-Embedding Predictive Architecture: predict the future in feature space instead of predicting pixels.

DINO-WM
The world model we build on (Zhou et al., ICML 2025): DINOv2 encoder + learned predictor + CEM planner.

Frozen
Not retrained. We use the published trained models as they are.

CEM (cross-entropy method)
Search procedure: sample 300 action sequences, keep the best 30, sample new ones around them, repeat 30 times.

MPC (model-predictive control)
Plan, execute only the first part of the plan, look again, plan again.

Round
One plan-and-execute cycle. We allow up to three.

Episode / start–goal pair
One attempt: a starting state and a goal image.

Seed
A number that fixes which episodes are picked. Several seeds show a result is not luck.

Success rate
Fraction of episodes in which the simulator says the goal was reached.

Trust score
A number for each imagined step: higher means less trustworthy.
Trust region
The set of situations where the world model's predictions are reliable.
Gate
The part that removes plans with an untrusted step.
Fallback ladder
What happens when too few plans pass: search wider, then plan shorter, then take a safe action.
Dropout
During training, parts of a network are randomly switched off to prevent overfitting.
MC-dropout
Keep dropout on at prediction time and predict several times (we use 4). Disagreement = uncertainty.
Action / image perturbation
Change the actions or the image slightly and see how much the prediction changes.
Distance to training data
How far an imagined state is from states the model saw during training.
Imagined cost
The planner's own estimate of how far a plan ends from the goal.
Baseline
A simple method that a new method must beat. Ours: random, and "distance moved".
Distance moved
How far the imagined state is from the start. Plans that change a lot are wrong more often.
True error
Difference between the imagined latent state and the real one, obtained by running the plan in the simulator.
AUROC
If you pick one wrong step and one correct step at random, the probability that the score ranks the wrong one higher. 0.5 = guessing, 1.0 = perfect.
95% interval
The range in which the true value lies with 95% confidence.
Bootstrap
Compute the result many times on random resamples of the data to see how much it varies.
Oracle
A cheating reference that knows the true error. Shows the best any trust score could do.
Push-T, Wall, PointMaze
Three simulated tasks: push a T-shaped block onto a target; walk through a door; roll a ball through a maze.

5. What each figure and table says

Fig. 1 (failure example)
Top row is reality, bottom row is imagination. The model imagines success; reality fails. This is the problem.
Fig. 2 (architecture)
Top: one planning round. Grey boxes are DINO-WM (frozen), blue boxes are what we add (trust scores and gate). Bottom: how we measured the true error.
Algorithm 1
The planning loop with our two extra lines: score every step, keep only trusted plans.
Table I and Fig. 3
Our success rates next to the paper's. They match after replanning.
Fig. 4
How the plan improves over the 30 CEM iterations. Most of the gain is in the first 10.

Table II and Fig. 5
How well each trust score detects wrong predictions. MC-dropout is clearly best; the dots are the values, the lines are the 95% intervals.
Table III
Success of the chosen plan when 30% of plans are removed by each score. All values are close, even the oracle.

6. Likely questions and answers

Q: What is new in your project? Isn't this just DINO-WM?

DINO-WM plans but never checks whether its imagination is correct. We add a reliability check for every imagined step, we built a way to measure the true error of each imagined step, and we found which check works (MC-dropout, AUROC 0.86). We also showed where such a check must be applied. The reproduction is the foundation; the trust scores and their evaluation are ours.

Q: Why is this a reinforcement learning project? Where is the reward?

It is model-based RL. The agent acts in an environment (the simulator), states change after actions, and the goal is reached only after several steps. Instead of a hand-written reward, the planner uses the distance between the imagined final state and the goal image as its cost. The success signal is only used for evaluation.

Q: What are the state, action and transition in your problem?

State: the camera image and the agent's position, turned into a latent feature vector. Action: a small movement of the agent. Transition: the simulator's physics. The world model is a learned approximation of this transition.

Q: Why did you not use PPO, SAC or DQN?

Those learn a policy by trial and error with a reward. Our setting has no reward and uses fixed, pre-recorded data. Planning with a world model needs no reward and keeps an explicit set of candidate plans, which is exactly what a trust check can filter.

Q: Why did you not train your own world model?

Training DINO-WM needs far more GPU time than we have. Using the published trained model also makes our results comparable with the paper, and it shows that SCoPe works on top of a frozen model without retraining.

Q: What is MC-dropout and why does it work?

Dropout randomly switches off parts of the network. If we keep it on and predict four times, we get four slightly different predictions. Where the model has learned the situation well, the four agree. Where it has not, they disagree. The disagreement is our trust score.

Q: What does an AUROC of 0.86 mean?

If you pick one wrong prediction and one correct prediction at random, MC-dropout gives the wrong one the higher score 86% of the time. Random guessing gives 50%.

Q: Why do you compare with "distance moved"? Why not only random?

Plans that change the scene a lot are naturally harder to predict, so even this trivial score reaches 0.79. A trust score that only beats random guessing could just be measuring how much the plan moves. MC-dropout beats this baseline too, on every seed, so it measures something real.

Q: Removing untrusted plans did not improve success. Did the project fail?

No. The test showed that even an oracle, which knows the true errors, does not improve success when plans are removed only at the final choice, because the plan with the lowest cost is rarely a badly predicted one. So the detection works, and the test tells us where to use it: inside every CEM iteration and every planning round, which is how Algorithm 1 is designed.

Q: Did you run the gate inside the full planner?

Answer honestly: "We evaluated the trust scores and the gate on recorded plans, where we know the true error. Running the gate inside every CEM iteration over several rounds on all three tasks is the next step and is listed as future work."

Q: Why is your Wall result with a single plan lower than the paper (0.52 vs 0.74)?

It is consistent over three seeds, so it is not luck. The most likely reason is that the published Wall setting stops CEM after only 10 iterations. After replanning we reach 0.95, the same as the paper's 0.96.

Q: How did you make it 3.8 times faster? Did it change the results?

We removed repeated work, for example encoding the shared starting image once instead of 300 times, and used faster GPU settings. We checked that the planner picks exactly the same plans as the original code. We rejected one faster option (bf16 precision) because it changed the plans.

Q: How did you get the true error?

For each recorded plan, we imagined it with the world model and also executed it in the simulator from the same starting state. We encoded the real images with the same encoder and compared imagined and real features at each step.

Q: Why the top 10%? Why per step?

We are interested in the worst predictions, the ones that can mislead the planner. Errors naturally grow with the number of steps ahead, so we mark the top 10% separately for each step; otherwise a score could look good just by flagging later steps.

Q: Why did image perturbation not work?

The image encoder is robust to small changes in the input image, so small noise barely changes the features. The test then measures the encoder, not the world model's prediction.

Q: Are your results statistically reliable?

We used three seeds for Push-T and Wall and 80 start–goal pairs for the trust scores, and we report 95% intervals. MC-dropout's whole interval (0.84–0.88) is above the baseline's (0.76–0.82), and it wins on each seed separately.

Q: Why only one seed for PointMaze?

Each run takes several hours on our GPU. PointMaze already reached 100% success on the first seed, so we spent the time on Wall and on the trust scores instead.

Q: How long do the experiments take?

One Push-T seed (50 episodes) takes about 3 to 4.5 hours. Measuring the true error takes about two minutes per start–goal pair.

Q: What are the limitations?

All tasks are simulated. The trust scores were measured on Push-T. Planning takes minutes per decision, so it is not real-time. A real robot would need a world model trained on robot data, such as V-JEPA 2-AC.

Q: Is "trust region" the same as in TRPO?

No. In TRPO it limits how much a policy changes during training. Here it means the situations in which the world model's predictions can be trusted.

Q: What would you do next?

Use the trust check inside every CEM iteration on all three tasks, combine several scores, and test under changed visual conditions such as different lighting or colours.

7. Things to avoid saying

- Do not say the gate improved success. Say: the trust score detects wrong predictions; the gate test shows where the check must be applied.
- Do not say "we trained the model". Say: we used the published trained DINO-WM models, unchanged.
- If you do not know an answer, say what you do know and offer to show the code or the logs in the repository.